

교과목 2 : 데이터 분석과 머신러닝 / 딥러닝

단원 1 : 데이터 분석

DAY1. 데이터 분석 개념 및 라이브러리

목 차

1. <u>데이터 분석 개요</u>		3
2. <u>NumPy 기본 1차원 배열</u>		13
3. <u>NumPy Narray 다차원 배열</u>		17
4. <u>NumPy 배열 연산</u>		20
5. <u>NumPy 함수</u>		23
6. <u>NumPy 데이터 처리</u>		29
7. <u>NumPy 데이터 통계</u>		36
8. <u>NumPy 행렬과 벡터 활용</u>		47

1. 데이터 분석 개요

데이터 분석(Data Analysis)은 수집된 데이터를 탐색하고 가공하여 의미 있는 정보와 패턴을 발견하고, 의사결정에 활용하는 과정이다.

즉, 단순한 데이터(Data)를 가치 있는 정보(Information)와 지식(Knowledge)으로 변환하는 활동이다.

데이터 분석의 목적

데이터 분석의 주요 목적은 다음과 같다.

- 문제 해결
- 미래 예측
- 의사결정 지원
- 업무 효율 향상
- 고객 행동 분석
- 이상 탐지
- 새로운 가치 발견

데이터 분석의 필요성

현대 사회에서는 대량의 데이터가 생성된다.

예시:

- SNS 데이터
- 쇼핑 데이터
- IoT 센서 데이터
- 금융 거래 데이터
- 웹 로그 데이터
- AI 서비스 데이터

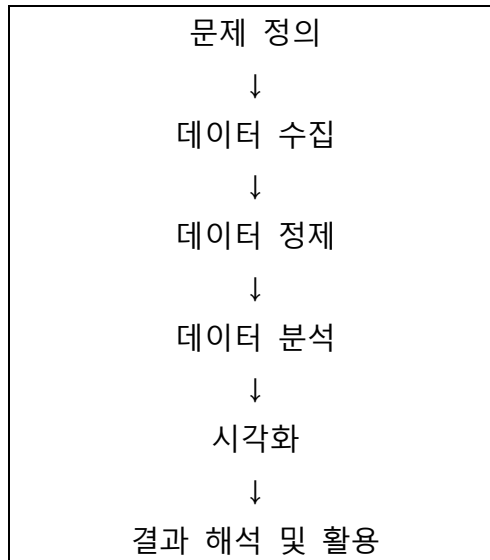
이러한 데이터를 분석하면:

- 고객 성향 파악
- 판매 예측
- 추천 시스템 구축
- 위험 탐지
- 자동화 시스템 구현

등이 가능하다.

데이터 분석의 기본 과정

일반적인 데이터 분석 과정은 다음과 같다.



1. 문제 정의

무엇을 분석할 것인지 결정하는 단계이다.

예시

- 고객 이탈 원인 분석
- 매출 감소 원인 분석
- 학생 성적 예측

2. 데이터 수집

분석에 필요한 데이터를 모으는 단계이다.

데이터 수집 방법

- DB(MySQL, Oracle)
- CSV/Excel 파일
- 웹 크롤링
- API
- 센서 데이터
- 로그 데이터

3. 데이터 정제(Data Cleaning)

오류 데이터나 불필요한 데이터를 처리하는 단계이다.

주요 작업

- 결측치 처리
- 이상치 제거
- 중복 제거
- 데이터 형식 통일

4. 데이터 분석

데이터의 패턴과 특징을 분석하는 단계이다.

분석 방법

(1) 기술 분석

현재 상태 분석

예:

- 평균
- 합계
- 분산

(2) 진단 분석

원인 분석

예:

- 왜 매출이 감소했는가?

(3) 예측 분석

미래 예측

예:

- 다음 달 판매량 예측

(4) 처방 분석

최적 행동 제안

예:

- 어떤 상품을 추천할 것인가?

5. 데이터 시각화

그래프와 차트를 사용하여 데이터를 이해하기 쉽게 표현한다.

사용 도구

- Matplotlib
- Seaborn
- Plotly
- Tableau
- Power BI

6. 결과 해석 및 활용

분석 결과를 실제 업무나 서비스에 적용한다.

예시

- 마케팅 전략 수립
- AI 모델 개선
- 추천 시스템 적용
- 이상탐지 시스템 운영

데이터 분석의 종류

1. 정형 데이터 분석

표 형태 데이터 분석

예:

- DB 테이블
- Excel

2. 비정형 데이터 분석

형태가 일정하지 않은 데이터 분석

예:

- 이미지
- 음성
- 영상
- SNS 텍스트

3. 반정형 데이터 분석

일부 구조를 가진 데이터

예:

- JSON
- XML

데이터 분석에 사용되는 기술

프로그래밍 언어

- Python
- R
- SQL

데이터 처리 라이브러리

- Pandas
- NumPy

머신러닝

- Scikit-learn
- TensorFlow
- PyTorch

데이터베이스

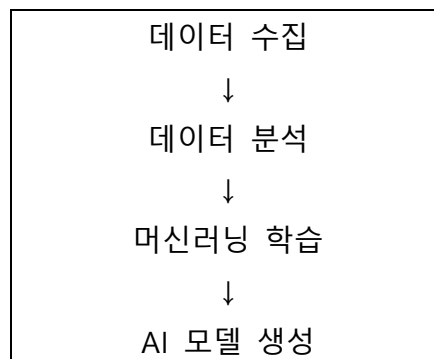
- MySQL
- MongoDB

시각화

- Matplotlib
- Seaborn

데이터 분석과 AI의 관계

데이터 분석은 AI와 머신러닝의 핵심 기반이다.



좋은 데이터 분석이 좋은 AI 성능을 만든다.

데이터 분석 프로세스

CRISP-DM은 **Cross Industry Standard Process for Data Mining**의 약자로, 데이터 분석 및 데이터 마이닝 프로젝트를 체계적으로 수행하기 위한 국제 표준 프로세스이다.

기업, 연구기관, AI/머신러닝 프로젝트 등에서 가장 널리 사용되는 데이터 분석 방법론 중 하나이며, 데이터 기반 문제 해결 과정을 단계별로 정의한다.

CRISP-DM의 특징

- 산업 분야와 관계없이 공통적으로 사용 가능
- 반복적(Iterative) 구조
- 데이터 분석뿐 아니라 AI/ML 프로젝트에도 활용
- 비즈니스 목표 중심 접근
- 실제 현업 데이터 분석 프로세스와 유사

CRISP-DM 전체 단계

CRISP-DM은 총 6 단계로 구성된다.

1. Business Understanding
2. Data Understanding
3. Data Preparation
4. Modeling
5. Evaluation
6. Deployment

1. Business Understanding (비즈니스 이해)

분석 프로젝트의 목적과 해결해야 할 문제를 정의하는 단계이다.

주요 활동

- 프로젝트 목표 정의
- 비즈니스 문제 분석
- 성공 기준 설정
- 데이터 분석 방향 결정

예시

온라인 쇼핑몰에서:

- 고객 이탈률 감소
- 매출 증가
- 추천 시스템 구축

등의 목표를 설정한다.

핵심 질문

- 무엇을 해결할 것인가?
- 왜 분석이 필요한가?
- 성공 기준은 무엇인가?

2. Data Understanding (데이터 이해)

수집된 데이터를 탐색하고 데이터의 구조와 품질을 파악하는 단계이다.

주요 활동

- 데이터 수집
- 데이터 구조 확인
- 데이터 시각화
- 이상치/결측치 탐색
- 데이터 패턴 분석

사용 기술

- 통계 분석
- EDA(Exploratory Data Analysis)
- 시각화

예시

- 고객 연령 분포 확인
- 구매 패턴 분석
- NULL 데이터 탐색

3. Data Preparation (데이터 준비)

모델링에 사용할 데이터를 정제하고 가공하는 단계이다.

실제 프로젝트에서 가장 많은 시간이 소요되는 단계이다.

주요 활동

- 결측치 처리
- 이상치 제거
- 데이터 정규화
- Feature Engineering
- 데이터 통합
- 학습용/테스트용 데이터 분리

중요성

Garbage In, Garbage Out(GIGO)!

즉, 입력 데이터 품질이 낮으면 결과도 좋지 않다.

4. Modeling (모델링)

데이터를 기반으로 머신러닝 또는 통계 모델을 생성하는 단계이다.

주요 활동

- 알고리즘 선택
- 모델 학습
- 파라미터 튜닝
- 모델 성능 비교

사용 알고리즘 예시

- 선형 회귀
- 의사결정트리
- 랜덤포레스트
- SVM
- 딥러닝
- 클러스터링

5. Evaluation (평가)

생성된 모델이 실제 비즈니스 목적에 적합한지 검증하는 단계이다.

주요 활동

- 정확도 평가
- 성능 검증
- 과적합 여부 확인
- 비즈니스 목표 충족 여부 판단

평가 지표 예시

- Accuracy
- Precision
- Recall
- F1-Score
- RMSE

핵심 질문

- 모델이 실제로 유용한가?
- 목표를 달성했는가?

6. Deployment (배포)

완성된 모델을 실제 서비스나 시스템에 적용하는 단계이다.

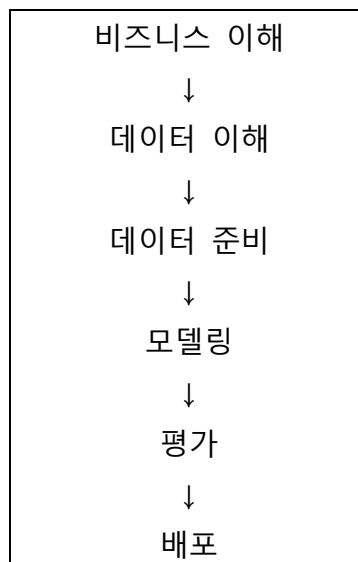
주요 활동

- 시스템 배포
- API 서비스화
- 모델 모니터링
- 유지보수
- 성능 개선

예시

- 웹 서비스 추천 시스템 적용
- AI 챗봇 운영
- 이상탐지 시스템 적용

CRISP-DM 프로세스 흐름



하지만 실제로는 순차적으로만 진행되지 않는다.

예를 들어:

- 모델 성능이 낮으면 다시 데이터 준비 단계로 이동
- 데이터 품질 문제가 발견되면 데이터 이해 단계로 복귀

하는 반복 구조를 가진다.

CRISP-DM 의 장점

1. 체계적 분석 가능

분석 과정을 단계별로 관리 가능

2. 프로젝트 성공률 향상

목표 중심 접근 가능

3. 다양한 산업 적용 가능

금융, 제조, 의료, AI 서비스 등 활용 가능

4. 재사용 가능

프로세스를 표준화 가능

CRISP-DM 의 단점

1. 반복 작업 많음

데이터 수정 및 재분석 반복 가능

2. 대규모 AI 프로젝트에는 세부 부족

딥러닝 MLOps 환경에서는 추가 구조 필요

3. 배포 이후 운영 관리 부족

현대 AI 시스템에서는 MLOps 와 함께 사용 필요

AI/머신러닝에서의 활용

CRISP-DM 은 현재 다음 분야에서 많이 활용된다.

- 머신러닝 프로젝트
- 데이터 분석 프로젝트
- 추천 시스템
- 이상탐지 시스템
- 고객 이탈 예측
- AI 서비스 개발
- 빅데이터 분석

CRISP-DM 은 데이터 분석 프로젝트를 체계적으로 수행하기 위한 대표적인 표준 방법론이며, 현재 AI 및 머신러닝 프로젝트에서도 매우 중요하게 사용되는 프로세스이다.

2. NumPy 기본 1차원 배열

1) NumPy 란?

- ♦ 2005 년에 Travis Oliphant 가 발표한 수치해석용 Python 패키지(라이브러리)
- ♦ 다차원의 행렬 자료구조인 NDArray 를 지원하여 벡터와 행렬을 사용하는 선형대수 계산에 주로 사용
- ♦ C 로 구현된 CPython 에서만 사용할 수 있으며 Jython, IronPython, PyPy 등의 Python 구현에서 사용할 수 없음
- ♦ C 로 구현된 내부 반복문을 사용하므로 Python 반복문과 비교해 속도가 빠름
- ♦ 행렬 인덱싱(Array Indexing)을 사용한 질의(Query) 기능을 이용하여 짧고 간단한 코드로 복잡한 수식 계산 가능

NumPy 설치하기

```
pip install numpy
```

pip 업그레이드하기

```
python -m pip install --upgrade pip
```

NumPy 사용

```
Import numpy as np
```

2) NumPy 내 함수의 특징

(1) 동일한 NumPy 모듈의 함수와 Nddarray 의 메서드 지원

- ♦ NumPy 모듈의 함수와 Nddarray 클래스의 메서드 이름이 같은 경우가 많으며 처리 규칙도 거의 같음

(2) 유니버설 함수(universal function: 범용 함수) 제공

- ♦ 실수, 복소수, 복소수 행렬(complex matrix)의 원소 간 연산(벡터화 연산)을 지원하는 함수
- ♦ 배열 간 또는 배열과 상수 간 연산을 지원
- ♦ ufunc 클래스를 이용하여 만들
예 : np.sin(), np.cos(), np.exp()와 같은 다양한 수학 함수들

3) NumPy 배열의 특징

- (1) NumPy 배열은 고정된 크기를 가짐
- (2) NumPy 배열의 요소들은 모두 같은 자료형과 메모리를 가짐
단, 객체 배열의 경우는 서로 다른 크기를 가질 수 있음
- (3) NumPy 배열 원소의 개수를 바꿀 수 없음
바꿀 때에는 기존 배열을 지우고, 배열을 새로 생성해야 함
- (4) 다차원 배열 혹은 행렬 연산이 가능함
- (5) Python 내장 시퀀스형에 비해 사용이 효율적이고, 코드가 간편함
- (6) Python 기반 과학 및 수학 연산 패키지에 활용됨

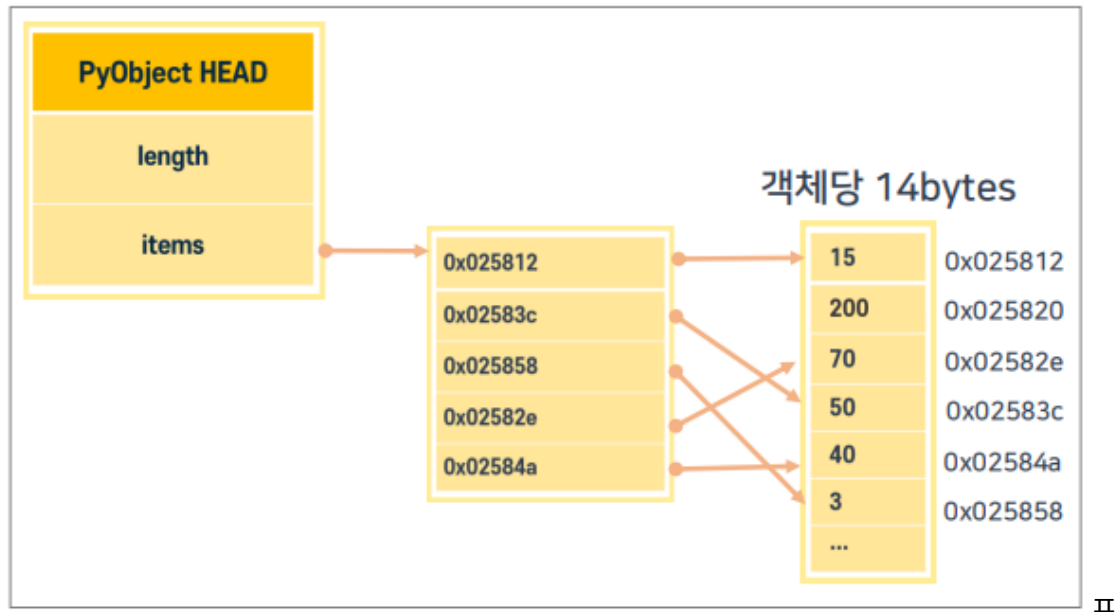
4) 1 차원 배열의 개요

- (1) 데이터가 한 줄로 나열된 형태
- (2) Python 에는 배열이 없음
- (3) 순차적으로 나열하는 배열 형태의 리스트 혹은 튜플이 있음
- (4) 리스트는 []로 표시하지만, 튜플은 ()로 표시함
- (5) 배열의 요소는 ,(콤마)로 구분함
- (6) 배열의 인덱스는 0 부터 시작함

a=[15, 50, 3, 70, 40]				
15	50	3	70	40
a[0]	a[1]	a[2]	a[3]	a[4]

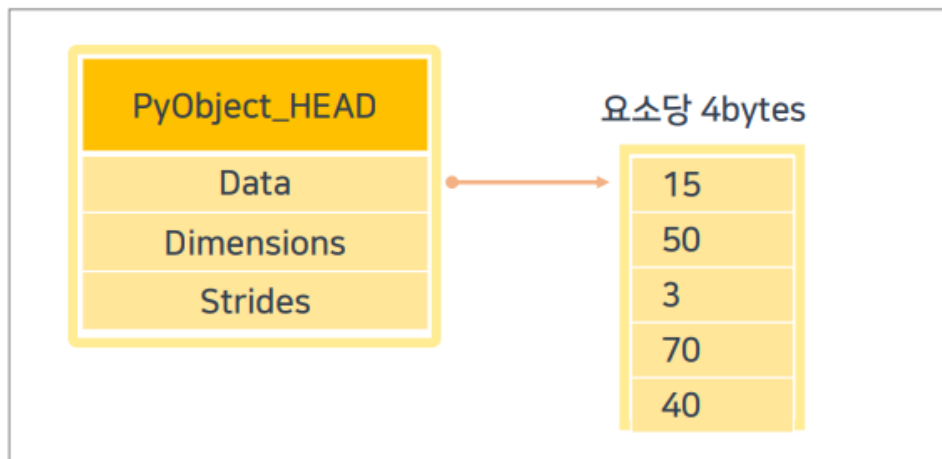
Python 리스트 구조 이해하기

- (1) Python 의 모든 객체를 저장할 수 있음
- (2) 요소 수정, 추가, 삭제가 가능함
- (3) 실제 값을 저장하는 것이 아니라, 해당 객체의 레퍼런스를 관리함



NumPy 배열 구조 이해하기

- (1) 같은 유형의 데이터만 저장할 수 있음
- (2) 원소의 개수를 바꿀 수 없음
- (3) array 클래스를 이용하여 생성함
- (4) Narray(다차원 배열) 타입을 가짐
- (5) 배열의 차원을 축(axis)이라 하며, 축의 개수를 "rank"라고 부름



5) array 를 이용한 1 차원 배열 생성하기

```
Import numpy as np
배열변수명 = np.array(리스트)
```

- (1) NumPy 에 속한 array 클래스를 이용하여 생성함
- (2) 동일한 타입의 값만 저장할 수 있음
- (3) 크기를 변경할 수 없음
- (4) Nddarray(다차원 배열) 데이터 타입을 가짐
- (5) **x.dtype**
배열 x 의 자료형을 알려줌
- (6) **x.ndim** 혹은 **np.ndim(x)**
배열 x 의 축의 개수를 알려줌
- (7) **x.shape** 혹은 **np. shape(x)**
배열 x 의 차원의 크기를 튜플 형식으로 알려 줌
- (8) **x.size** 혹은 **np. size(x)**
배열 x 의 원소의 개수를 알려줌
- (9) NumPy 의 데이터 타입
dtype 명령을 이용하여 타입을 확인할 수 있음

타입	설명
bool	불리언(True 또는 False)
int8	8비트 정수(-128 ~ 127)
int16	16비트 정수(-32768 ~ 32767)
int32	32비트 정수(-2147483648 ~ 2147483647)
int64	64비트 정수(-9223372036854775808 ~ 9223372036854775807)
uint8	8비트 부호 없는 정수(0 ~ 255)
uint16	16비트 부호 없는 정수(0 ~ 65535)
uint32	32비트 부호 없는 정수(0 ~ 4294967295)
uint64	64비트 부호 없는 정수(0 ~ 18446744073709551615)
float16	16비트 실수, 반정밀도 실수
float32	32비트 실수, 단정밀도 실수
float64	64비트 실수, 배정밀도 실수
complex64	복소수, 32비트 실수 2개로 실수와 허수로 나타냄
complex128	복소수, 64비트 실수 2개로 실수와 허수로 나타냄

3. NumPy Narray 다차원 배열

1) Narray 구조 이해하기

- (1) 데이터가 여러 줄로 나열된 2 차원 이상의 배열
- (2) 자료형이 같은 원소들로 구성됨
- (3) 배열의 요소는 ,(콤마)로 구분
- (4) array 클래스를 이용하여 생성
- (5) 배열의 인덱스는 0 부터 시작하며, 정수형 튜플임
- (6) 차원을 축(axis)이라 하며, 축의 개수를 rank 라 부름

	1열	2열	3열	← 제1축
1행	배열 [0][0]	배열 [0][1]	배열 [0][2]	← 제2축
2행	배열 [1][0]	배열 [1][1]	배열 [1][2]	

2) NumPy Narray 기본

- (1) NumPy Narray 란?
동일한 유형의 데이터를 저장하기 위한 다차원 배열
- (2) Narray 생성하기
array 를 이용하여 Narray 생성

```
Narray 객체명 = np.array(리스트)
```

- (3) 2 차원 배열 생성하기

```
[형식] Narray객체명
      = np.array([[요소1, 요소2],
                  [요소3, 요소4],
                  [요소5, 요소6]])
```

6 개의 요소를 갖는 3 행 2 열 2 차원 배열이 생성됨

3) Narray 생성 함수

- ♦ array(리스트)
주어진 리스트를 Narray 형태로 바꾸어 줌

데이터 분석

예 : `arr = np.array([1,2,3,4])`

1	2	3	4
<code>arr[0]</code>	<code>arr[1]</code>	<code>arr[2]</code>	<code>arr[3]</code>

♦ `arrange(시작 값, 끝 값[, 간격], dtype=None)`

시작 값에서 끝 값이 될 때까지 1 씩 증가시키면서 Narray 를 생성

예 : `arr=np. arrange(3, 7)`

3	4	5	6	7
<code>arr[0]</code>	<code>arr[1]</code>	<code>arr[2]</code>	<code>arr[3]</code>	<code>arr[4]</code>

♦ `empty((행, 열))`

주어진 행과 열 개수를 갖는 2 차원 배열을 생성

예 : `arr=np. empty((2, 3))`

♦ `ones((행, 열))`

주어진 행과 열 개수를 갖는 2 차원 배열을 생성(값은 1 을 할당)

예 : `arr=np. ones((2, 3))`

♦ `zeros((행, 열))`

주어진 행과 열 개수를 갖는 2 차원 배열을 생성(값은 0 을 할당)

예 : `arr=np. zeros((2, 3))`

	1열	2열	3열
1행	<code>arr[0][0]</code>	<code>arr[0][1]</code>	<code>arr[0][2]</code>
2행	<code>arr[1][0]</code>	<code>arr[1][1]</code>	<code>arr[1][2]</code>

♦ `full((행, 열), 값)`

주어진 행과 열 개수를 갖는 2 차원 배열을 생성(주어진 값을 할당)

예 : `arr=np .full((2, 3),10)`

♦ `eye(x)`

x 행, x 열의 단위 행렬을 생성(대각선 값은 1 을 할당, 나머지는 0)

예 : `arr=np .eye(2)`

	1열	2열
1행	<code>arr[0][0]</code>	<code>arr[0][1]</code>
2행	<code>arr[1][0]</code>	<code>arr[1][1]</code>

1의 값을 할당,
나머지는 0을 할당

4) Narray 관련 함수

◆ 배열의 데이터 타입 확인

배열명.dtype

type(배열의 요소)

예 : arr.dtype

예 : type(arr[0])

◆ 배열의 데이터 타입 변경

배열명.astype(np.데이터 타입)

예 : arr.astype(np.float32)

◆ 배열 요소의 크기 확인

배열의 요소 혹은 배열명.itemsize

예 : arr.itemsize, arr[0].itemsize

◆ 배열 요소의 값 확인

배열명[x] : x 인덱스를 갖는 요소의 값을 읽어 옴

예 : arr[1] -> 배열 arr 의 두번째 요소의 값을 읽어 옴

◆ 배열의 차원(축) 확인 및 변경

확인: 배열명.shape

변경: 배열명.shape=행[, 열]

◆ resize 으로 배열의 차원을 변경

예 : arr.resize(2, 3) -> arr = np.array([1,2,3,4,5,6]) 1 차원 배열을 2 행 3 열의 2 차원 배열로 바꿈

주의!!!

5 개의 요소를 갖는 1 차원 배열을 2 × 3 배열로 변경하면 **ERROR** 발생

◆ reshape 으로 배열의 차원을 변경

[형식] np.reshape(변경할 배열, [[면,]행,] 열)

[형식] 배열.reshape([[면,]행,] 열)

4. NumPy 배열 연산

1) 배열과 스칼라 연산

◆ 스칼라 (Scalar) : 값 1 개

- (1) 배열의 각 요소의 스칼라 값을 각각 연산함
- (2) 정수와 실수 연산의 결과는 실수로 반환함

◆ 예 : 사칙연산(+, -, *, /)



2) 배열 인덱싱과 슬라이싱

- (1)인덱스 번호는 0 부터 시작(왼쪽에서 오른쪽 방향으로)
- (2)다차원 배열의 경우, 바깥부터 안쪽으로 하나씩 접근
- (3)배열의 슬라이싱 결과는 Narray
- (4)슬라이싱한 객체는 새로운 인덱스를 할당
- (5)슬라이싱한 객체에 특정 값을 대입하면 모두 해당 값으로 변경 (브로드캐스팅: broadcasting)
- (6)슬라이싱한 객체는 원본 배열의 뷰(View)
- (7)슬라이싱한 객체의 값을 변경하면 원본 배열의 값도 변경

	1열	2열	3열	
1행	배열[0][0] or 배열[0,0]	배열[0][1] or 배열[0,1]	배열[0][2] or 배열[0,2]	배열[0]or 배열[0,]
2행	배열[1][0] or 배열[1,0]	배열[1][1] or 배열[1,1]	배열[1][2] or 배열[1,2]	배열[1] or 배열[1,]
	배열[:0] X			

3) 슬라이싱 (Slicing) 수행

- ◆ 시작인덱스부터 끝인덱스-1 까지 슬라이싱

[형식] 배열명[시작인덱스:끝인덱스]

- ◆ 처음부터 지정한 인덱스까지 슬라이싱

[형식] 배열명[:끝인덱스+1] or 배열명[0:끝인덱스+1]

- ◆ 지정한 인덱스부터 끝까지 슬라이싱

[형식] 배열명[시작인덱스:] or 배열명[시작인덱스:-1]

4) 배열의 전치

- ◆ 배열의 행과 열, 즉 축을 바꾸는 것
- ◆ 1 차원 배열의 전치는 불가능
- ◆ 1 차원 배열을 전치하고자 하는 경우, 1 x N 형태의 2 차원 배열로 변환해야 함
- ◆ T 속성 혹은 transpose, swapaxes 함수를 이용

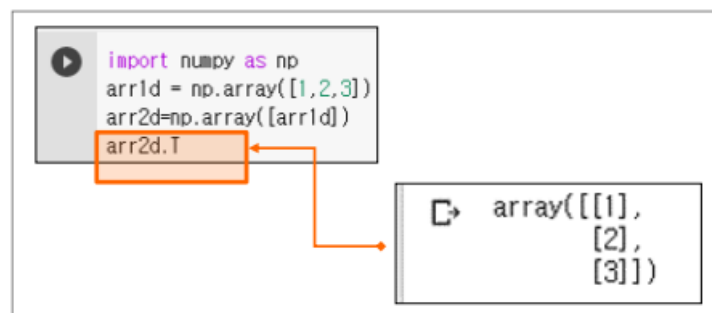


(1) 1 차원 배열의 축 바꾸기

- ① 1 차원 배열을 1 x N 형태의 2 차원 배열로 변환

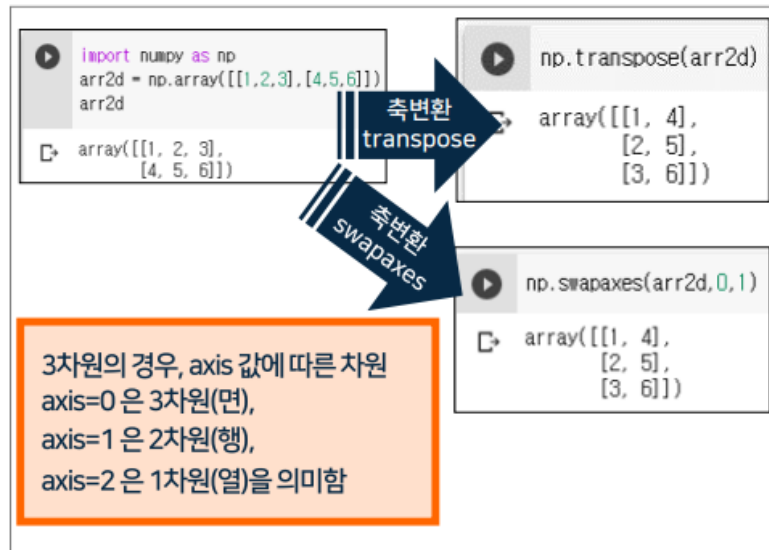


- ② 배열명.T 혹은 np.transpose(배열명), np.swapaxes(배열명, 0,1) 을 실행시켜 축을 바꾸어 줌



(2) 2 차원 배열의 축 바꾸기

① T 속성 혹은 transpose, swapaxes 함수로 축을 바꾸어 줌



5. NumPy 함수

1) 유니버설 함수

- (1) Nddarray 안의 데이터를 원소별로 연산을 수행해 주는 함수
- (2) 배열 간 또는 배열과 상수 간 연산을 지원함
- (3) 1 개의 인자를 받아 값을 반환하는 유니버설 함수를 단항 유니버설 함수, 2 개의 인자를 받아 값을 반환하는 유니버설 함수를 이항 유니버설 함수라고 함
- (4) 벡터화된 유니버설 함수(ufunc)는 브로드캐스팅의 함수형 버전임
- (5) ufunc 를 사용하면 함수를 한 번 불러서 배열의 모든 아이템에 적용할 수 있음

2) 유니버설 함수 종류

(1) 단항 유니버설 함수

함수명	설명
abs, fabs	각 원소의 절댓값을 구함. 복소수가 아닌 경우, fabs를 사용하면 빠른 연산이 가능함
sqrt	각 원소의 제곱근을 구함
square	각 원소의 제곱을 구함
exp	각 원소의 밑이 자연상수 e인 지수함수 값을 구함
sign	각 원소의 부호를 구함
ceil	각 원소의 값보다 같거나 큰 정수 중 가장 작은 값을 구함
floor	각 원소의 값보다 같거나 작은 정수 중 가장 큰 값을 구함
logical_not	수행 결과의 부정을 구함
rint	각 원소의 소숫자리를 반올림함
modf	각 원소의 몫과 나머지를 각각의 배열로 구함
isnan	각 원소가 NaN인지 아닌지를 각각의 배열로 구함(논릿값으로)
isfinite, isinf	각 원소가 유한한지, 무한한지를 각각의 배열로 구함(논릿값으로)
삼각함수	sin(), cos(), tan(), sinh(), cosh(), tanh()를 구함
역삼각함수	arccos(), arcsin(), arctan(), arccosh(), arcsinh(), arctanh()를 구함

(2) 이항 유니버설 함수

함수명	설명
round, around	배열의 각 요소를 소수점 이하 자리로 반올림함
add	두 배열의 같은 위치의 원소끼리 합함
subtract	두 배열의 같은 위치의 원소끼리 합함 끼리 빼
multiply	두 배열의 같은 위치의 원소끼리 곱함
divide, floor_divide	두 배열의 같은 위치의 원소끼리 나눔. floor_divide는 몫만 취함
power	첫 번째 배열의 원소를 두 번째 배열이 원소만큼 제곱함
maximum, fmax	각 배열의 두 원소 중 큰 값을 구함. fmax는 NaN을 무시함
minimum, fmin	각 배열의 두 원소 중 작은 값을 구함. fmin는 NaN을 무시함
mod	첫 번째 배열의 원소를 두 번째 원소로 나눈 나머지를 구함
대소 비교	greater, greater_equal, less, less_equal, equal, not_equal을 구함
논리연산	logical_or, logical_and를 구함

3) 반올림, 올림, 내림 익히기

(1) 반올림 익히기

- 소수점 자릿수에서 반올림하여 정수 구하기

① np.round(data, decimals)

```
import numpy as np
a=np.array([2.41, 2.73, 4.55, 4.84])
np.round(a)
```

② np.round(data, decimals)

```
import numpy as np
a=np.array([2.41, 2.73, 4.55, 4.84])
np.round(a)
```

- 소수점 첫째 자리까지 표시하기(반올림)

① np.round(data, decimals)


```
import numpy as np
a=np.array([2.41, 2.73, 4.55, 4.84])
np.round(a,1)
```

② np.around(data, decimals)

```
import numpy as np
a=np.array([2.41, 2.73, 4.55, 4.84])
np.around(a,1)
```

(2) 소수점 자릿수에서 올림하여 정수 구하기

① np.ceil(data)

```
import numpy as np
a=np.array([2.41, 2.73, 4.55, 4.84])
np.ceil(a)
```

(3) 소수점 자릿수에서 내림하여 정수 구하기

① np.floor(data)

```
import numpy as np
a=np.array([2.41, 2.73, 4.55, 4.84])
np.floor(a)
```

4) 논리함수 익히기

(1) or 연산하기: 두 배열 중 하나가 참이면 참

① np.logical_or(배열 1, 배열 2)

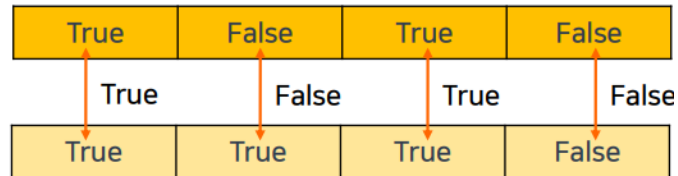
```
import numpy as np
a=np.array([True, False, True, False])
b=np.array([True, True, True, False])
np.logical_or(a,b)
```

True	False	True	False
True	True	True	False
True	True	True	False

(2) and 연산하기: 두 배열의 요소 모두 참이면 참

① np.logical_and(배열 1, 배열 2)

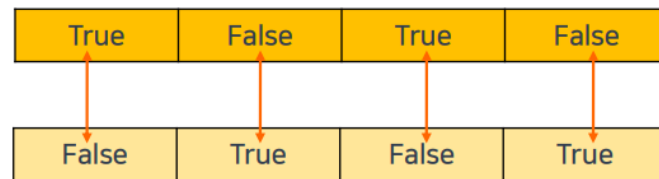
```
import numpy as np
a=np.array([True, False, True, False])
b=np.array([True, True, True, False])
np.logical_and(a,b)
```



(3) not 연산하기: 두 배열의 요소가 모두 참이면 참

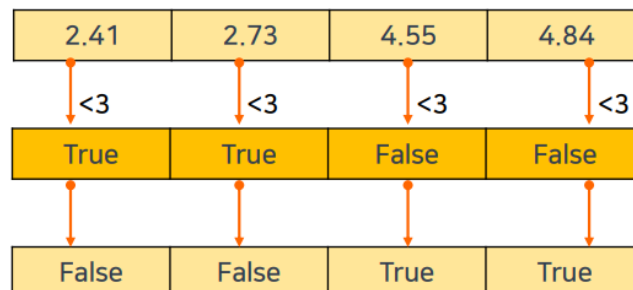
① np.logical_not(배열 또는 배열 비교연산식) 배열의 not 연산

```
import numpy as np
a=np.array([True, False, True, False])
np.logical_not(a)
```



② 배열 비교연산식의 not 연산

```
import numpy as np
a=np.array([2.41, 2.73, 4.55, 4.84])
np.logical_not(a<3)
```

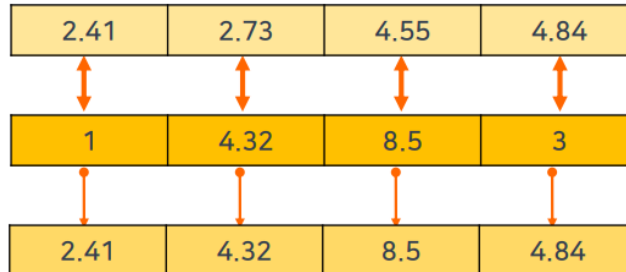


5) maximum 과 minimum 익히기

(1) maximum 을 이용하여 최댓값 구하기

① np.maximum(배열 1, 배열 2)

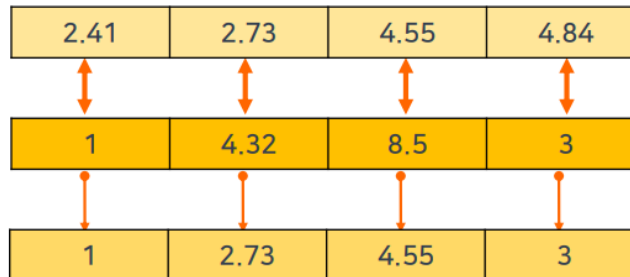
```
import numpy as np
a=np.array([2.41, 2.73, 4.55, 4.84])
b=np.array([1, 4.32, 8.5, 3])
np.maximum(a,b)
```



(2) minimum 을 이용하여 최솟값 구하기

① np.minimum(배열 1, 배열 2)

```
import numpy as np
a=np.array([2.41, 2.73, 4.55, 4.84])
b=np.array([1, 4.32, 8.5, 3])
np.minimum(a,b)
```



6) 배열 간 사칙 연산하기

(1) 1 차원 배열 간의 사칙 연산하기

```
import numpy as np
a=np.array([2.41, 2.73, 4.55, 4.84])
b=np.array([1, 4.32, 8.5, 3])
np.add(a,b)
```

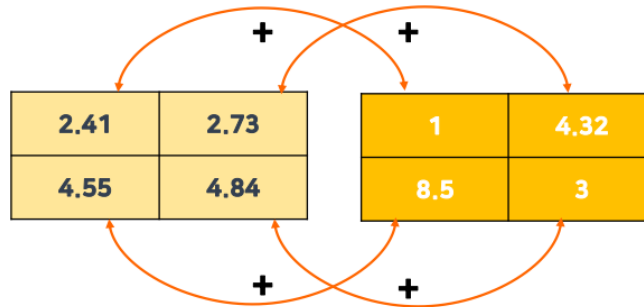
데이터 분석

2.41	2.73	4.55	4.84
↑ +	↑ +	↑ +	↑ +
1	4.32	8.5	3
↓	↓	↓	↓
3.41	7.05	13.05	7.84

7) 행렬 간 사칙 연산하기

(1) 2X2 행렬 간 합 연산하기

```
import numpy as np
a=np.array([2.41, 2.73, 4.55, 4.84]).reshape(2,2)
b=np.array([[1, 4.32], [8.5, 3]])
np.add(a,b)
```



6. NumPy 데이터 처리

배열을 사용한 데이터 처리

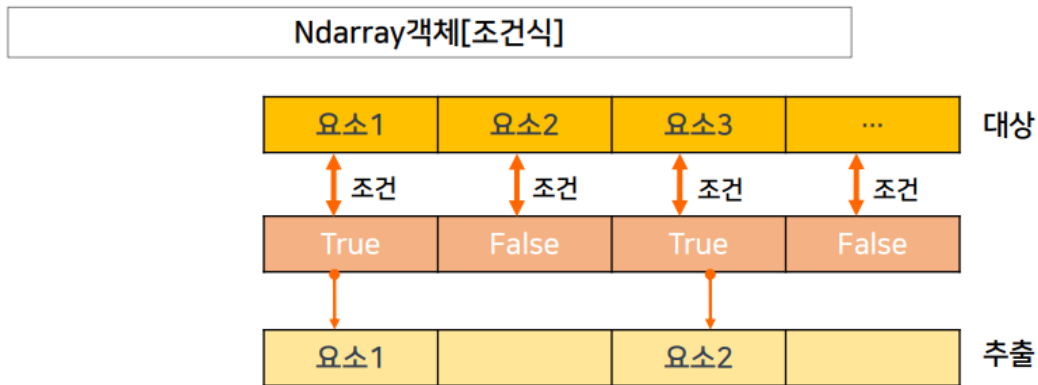
1. 배열 연산으로 조건절 표현하기

1) 논리식 검색

(1) 논리 인덱싱(Boolean indexing)이란?

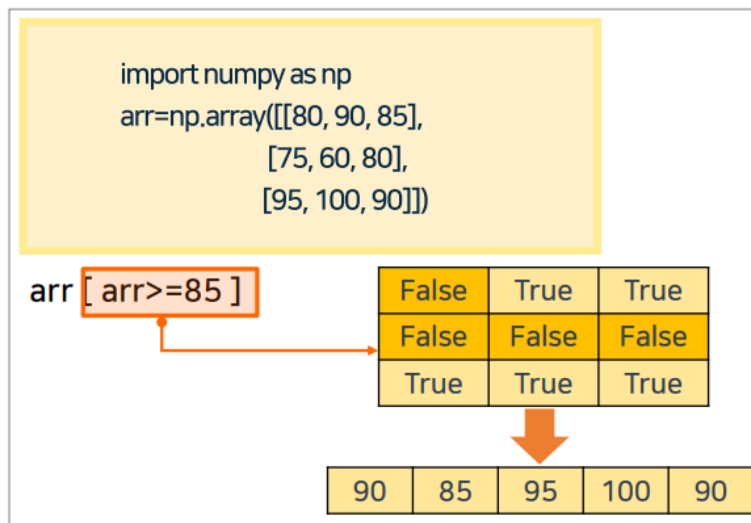
- 배열 각 요소의 선택 여부를 True, False 로 표현
- 대상 배열의 인덱스가 True 인 요소만 결과로 추출하고자 할 때 사용

(2) 논리 인덱싱 형식



(3) 논리 인덱싱 활용하기

- 주어진 배열에서 85 이상인 요소를 추출하기



2) where 함수

(1) where 함수란?

- 조건을 충족하는 배열의 색인을 생성

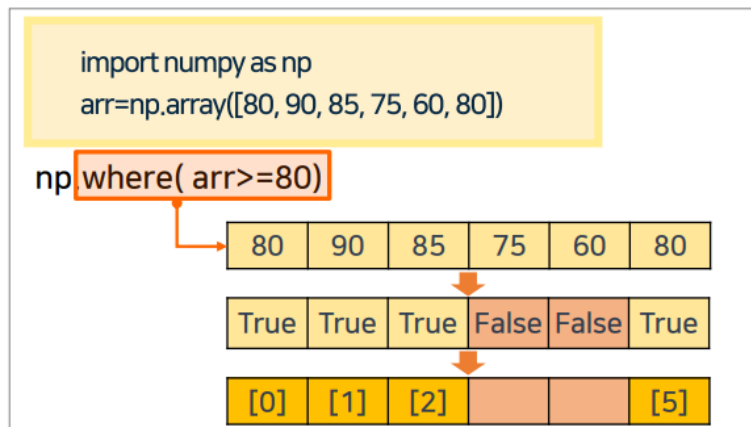
(2) where 함수 형식

```
numpy.where(조건식 [, x, y])
```

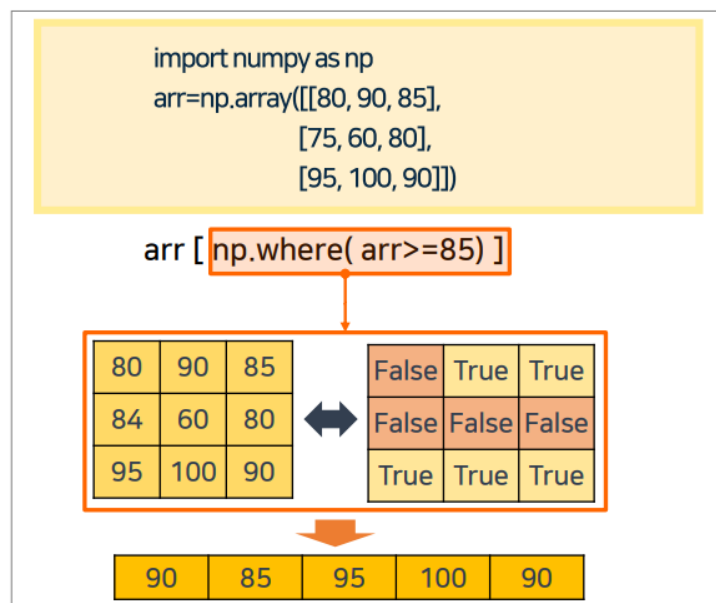
- x와 y는 없을 수도 있음
- 조건식만 명기하는 경우 색인만 반환
- x는 조건을 만족했을 때 반환할 값, y는 조건을 만족하지 않았을 때 반환할 값을 명기

(3) where 활용하기

- 주어진 배열에서 80 이상인 요소의 위치



- 주어진 배열에서 85 이상인 요소를 추출하기



2. 수학함수사용하기

1) 산술 함수

(1) 사칙연산 함수

- 사칙연산 함수 종류

함수	설명
<code>numpy.add(x,y)</code>	x와 y를 더한 결과값을 반환함
<code>numpy.subtract(x,y)</code>	x에서 y를 뺀 결과값을 반환함
<code>numpy.multiply(x,y)</code>	x에 y를 곱한 결과값을 반환함
<code>numpy.divide(x,y)</code>	x를 y로 나눈 결과값을 반환함
<code>numpy.mod(x,y)</code> <code>numpy.remainder(x,y)</code>	x를 y로 나눈 나머지를 반환함
<code>numpy.divmod(x,y)</code>	x를 y로 나눈 몫과 나머지를 반환함

- 사칙연산 함수 사용하기

- 두 배열의 덧셈 연산하기

```
import numpy as np
arr1 = np.array([10,20,30])
arr2 = np.array([4,5,6])
```

`arr1 + arr2` **=** `np.add(arr1, arr2)`

10	20	30	
+	+	+	
4	5	6	

➔

14	25	36
----	----	----

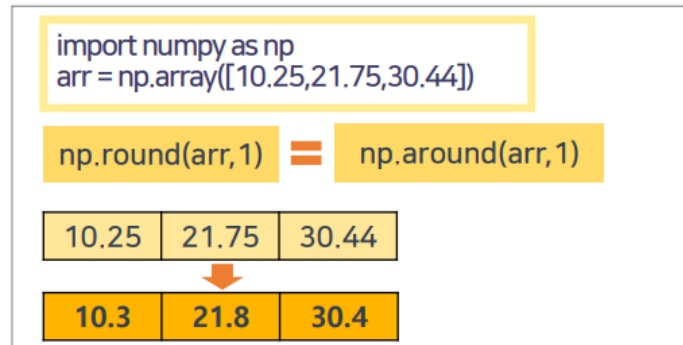
(2) 소수점 이하 처리 함수

- 소수점 이하 처리 함수 종류

함수	설명
<code>numpy.fix(x)</code>	0을 향해 가장 가까운 정수를 구함
<code>numpy.trunc(x)</code>	소수점 이하 자리를 버리고 정수를 만듦
<code>numpy.ceil(x)</code>	소수점 자리 수를 올림해서 정수로 만듦
<code>numpy.floor(x)</code>	소수점 자리 수를 버림해서 정수로 만듦
<code>numpy.round(x[,y])</code> <code>numpy.around(x[,y])</code>	x의 y+1 소수점 자리에서 반올림함

데이터 분석

- 소수점 이하 처리 함수 사용하기
 - 다음 아래 배열의 값을 소수 1 자리까지 표시하기(반올림)

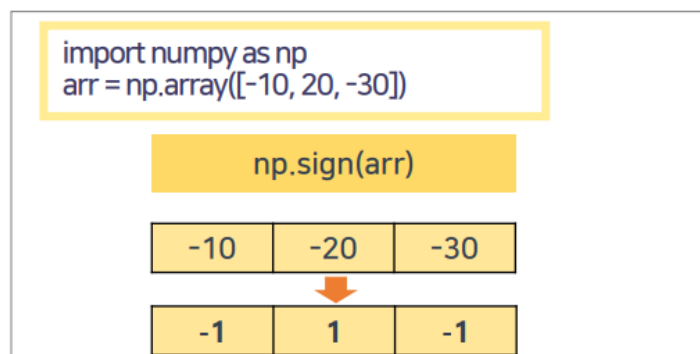


(3) 부호, 제곱, 제곱근 처리 함수

- 부호, 제곱, 제곱근 처리 함수 종류

함수	설명
<code>numpy.sign(x)</code>	x가 음수의 경우는 -1, 양수의 경우는 1을 반환
<code>numpy.abs(x)</code> <code>numpy.absolute(x)</code>	X가 음수일 경우, 양수로 변환하여 반환
<code>numpy.positive(x)</code>	x의 'numerical positive'를 반환(+x)
<code>numpy.negative(x)</code>	x의 'numerical negative'를 반환(-x)
<code>numpy.square(x)</code>	x의 제곱을 반환 (<code>numpy.power(x,2)</code> 와 동일)
<code>numpy. power(x, y)</code>	x의 y승 값을 반환

- 부호, 제곱, 제곱근 처리 함수 사용하기
 - 다음 아래 배열의 값을 부호를 출력하기



2) 삼각 함수

(1) 기본 삼각 함수

▪ 기본 삼각 함수 종류

함수	설명
<code>numpy.sin(x)</code>	x의 사인 함수의 값을 구함
<code>numpy.cos(x)</code>	x의 코사인 함수의 값을 구함
<code>numpy.tan(x)</code>	x의 탄젠트 함수의 값을 구함

▪ 기본 삼각 함수 사용하기

- 각도 0, 30, 45, 60, 90 의 사인 함수의 값을 구하기

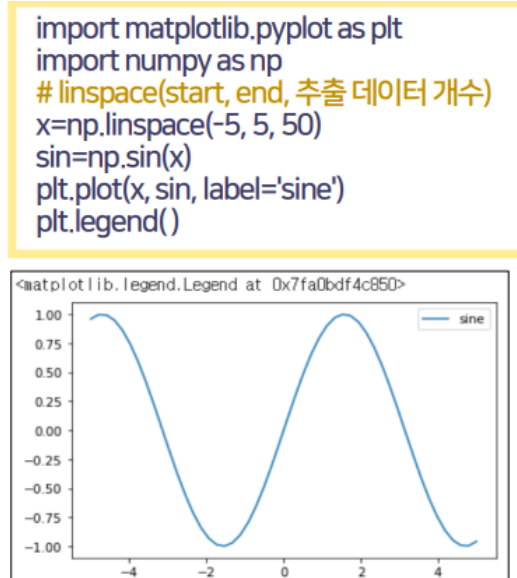
```
import numpy as np
arr = np.array([0., 30., 45., 60., 90.])
```

`np.sin( arr*np.pi/180. )` = `np.sin( np.radians(arr) )`

0.	30.	45.	60.	90.
0.	0.5	0.70710678	0.8660254	1.

▪ 기본 삼각 함수 그래프를 그려보기

- X 축 -5~5 사이에 50 개의 데이터를 발생시켜 사인 그래프를 그리기



(2) 역삼각 함수

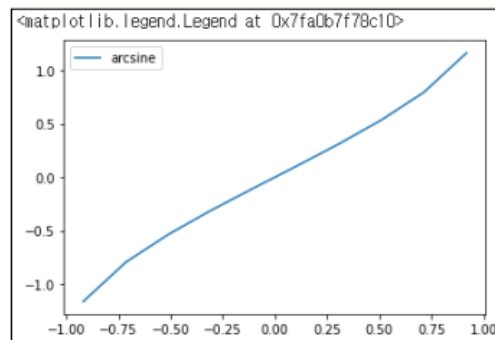
▪ 역삼각 함수 종류

함수	설명
<code>numpy.arcsin(x)</code>	x의 아크사인 함수의 값을 구함
<code>numpy.arccos(x)</code>	x의 아크코사인 함수의 값을 구함
<code>numpy.arctan(x)</code>	x의 아크탄젠트 함수의 값을 구함

▪ 역삼각 함수 그래프를 그리기

- X 축 -5~5 사이에 50 개의 데이터를 발생시켜 아크사인 그래프를 그리기

```
import matplotlib.pyplot as plt
import numpy as np
# linspace(start, end, 추출 데이터 개수)
x=np.linspace(-5, 5, 50)
arcsin=np.arcsin(x)
plt.plot(x, arcsin, label='arcsine')
plt.legend()
```



3. 불리언 배열을 위한 함수 사용하기

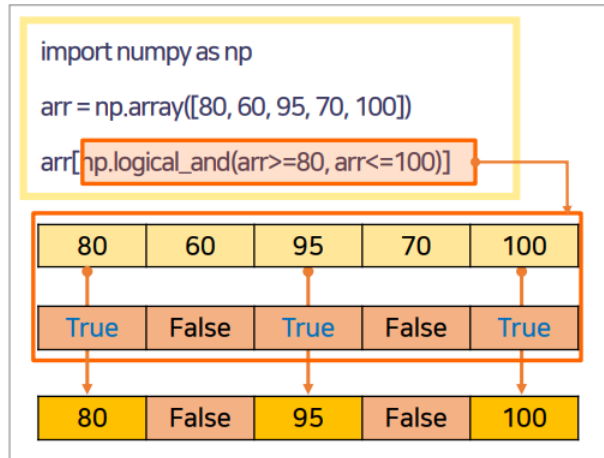
1) 논리 연산 함수

(1) 논리 연산 함수 종류

함수	설명
<code>numpy.logical_and(x,y)</code>	x와 y 모두 True이면 True, 그렇지 않으면 False를 반환
<code>numpy.logical_or(x,y)</code>	x 또는 y가 True이면 True, 그렇지 않으면 False를 반환
<code>numpy.logical_xor(x,y)</code>	x와 y가 서로 상반된 값을 가지면 True이면 True, 그렇지 않으면 False를 반환
<code>numpy.logical_not(x)</code>	X가 True이면 True, 그렇지 않으면 False를 반환

(2) 논리 연산 함수 사용하기

- 배열의 값이 80~100 인 값들을 구하기

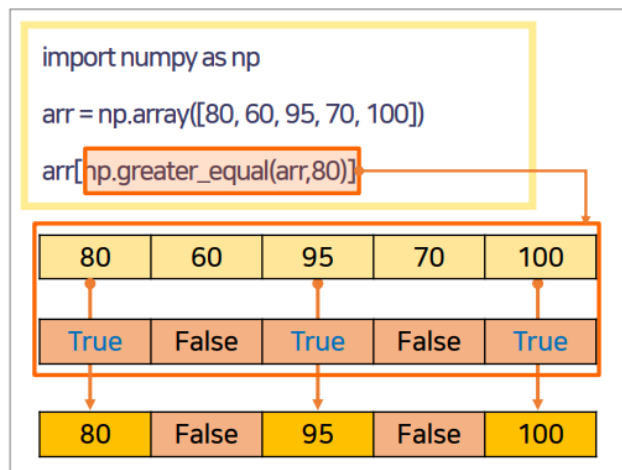


2) 비교 연산 함수

(1) 비교 연산 함수 종류

함수	설명
numpy.less(x, y)	x<y이면 True, 그렇지 않으면 False를 반환
numpy.greater(x, y)	x>y이면 True, 그렇지 않으면 False를 반환
numpy.equal(x, y)	x=y이면 True, 그렇지 않으면 False를 반환
numpy.less_equal(x, y)	x<=y이면 True, 그렇지 않으면 False를 반환
numpy.greater_equal(x, y)	x>=y이면 True, 그렇지 않으면 False를 반환
numpy.not_equal(x, y)	x<>y이면 True, 그렇지 않으면 False를 반환

- 배열의 값이 80 이상인 값들을 구하기



7. NumPy 데이터 통계

배열을 사용한 데이터 통계 처리하기

1. 통계 함수 사용하기

1) 최댓값, 최솟값 구하기

(1) 최댓값 관련 함수 | 최댓값 구하기

- 배열의 원소 중 최댓값을 구함
- 행 또는 열별 최댓값을 구할 수도 있음

[형식]

```
numpy.max(배열[, axis=0 또는 1 [, out=배열]])
```

- ✓ axis를 생략하면, 전체 요소 중 최댓값을 구함
- ✓ axis를 지정하면, 행 또는 열별 최댓값을 구함
- ✓ 0은 열별, 1은 행별 최댓값을 구함
- ✓ out에는 결과값을 저장하기 위한 배열을 지정할 수 있음

(2) 최댓값 관련 함수 | 최댓값의 위치 구하기

- 배열을 1차원으로 변경 후 최댓값의 위치를 구함
- 행 또는 열별 최댓값의 위치를 구할 수도 있음

[형식]

```
numpy.argmax(배열[, axis=0 또는 1 [, out=배열]])
```

- ✓ axis를 생략하면, 전체 요소 중 최댓값의 위치를 구함
- ✓ axis를 지정하면, 행 또는 열별 최댓값의 위치를 구함
- ✓ 0은 열별, 1은 행별 최댓값의 위치를 구함
- ✓ out에는 결과값을 저장하기 위한 배열을 지정할 수 있음

- 주어진 배열에서 행별 최댓값을 구함

```
import numpy as np
arr=np.array([[80,90,85],
              [75,60,80],
              [95,100,90]])
```

`np.max(arr, axis=1) → [90, 80, 100]`

	[0]	[1]	[2]	
[0]	80	90	85	→ 90
[1]	84	60	80	→ 80
[2]	95	100	90	→ 100

(3) 최솟값 관련 함수 | 최솟값 구하기

- 배열의 원소 중 최솟값을 구함
- 행 또는 열 별 최솟값을 구할 수도 있음

[형식]

`numpy.min(배열[, axis=0 또는 1, out=배열])`

- ✓ axis를 생략하면, 전체 요소 중 최솟값을 구함
- ✓ axis를 지정하면, 행 또는 열 별 최솟값을 구함
- ✓ 0은 열 별, 1은 행 별 최솟값을 구함
- ✓ out에는 결과값을 저장하기 위한 배열을 지정할 수 있음

(4) 최솟값 관련 함수 | 최솟값의 위치 구하기

- 배열을 1 차원으로 변경 후 최솟값의 위치를 구함
- 행 또는 열 별 최솟값의 위치를 구할 수도 있음

[형식]

`numpy.argmin(배열[, axis=0 또는 1, out=배열])`

- ✓ axis를 생략하면, 전체 요소 중 최솟값의 위치를 구함
- ✓ axis를 지정하면, 행 또는 열 별 최솟값의 위치를 구함
- ✓ 0은 열 별, 1은 행 별 최솟값의 위치를 구함
- ✓ out에는 결과값을 저장하기 위한 배열을 지정할 수 있음

2) 대표값 구하기

(1) 중앙값 구하기

- 원소의 개수가 홀수일 경우는 중간에 위치한 값을 반환
- 원소의 개수가 짝수일 경우는 중간에 위치한 2 개의 평균을 반환

[형식]

```
numpy.median(배열[, axis=0 또는 1 [, out=배열]])
```

- ✓ axis를 생략하면, 전체 요소 중 중앙값을 구함
- ✓ axis를 지정하면, 행 또는 열 별 중앙값을 구함
- ✓ 0은 열 별, 1은 행 별 중앙값을 구함
- ✓ out에는 결과값을 저장하기 위한 배열을 지정할 수 있음

(2) 평균값 관련 함수 | 평균값 구하기

- 배열의 원소의 평균값을 구함
- 행 또는 열 별 평균값을 구할 수도 있음

[형식]

```
numpy.mean(배열[, axis=0 또는 1 [, out=배열]])
```

- ✓ axis를 생략하면, 전체 요소의 평균값을 구함
- ✓ axis를 지정하면, 행 또는 열 별 평균값을 구함
- ✓ 0은 열 별, 1은 행 별 평균값을 구함
- ✓ out에는 결과값을 저장하기 위한 배열을 지정할 수 있음

(3) 평균값 관련 함수 | NaN 값을 무시하고 평균값 구하기

- 배열에서 NaN 이 아닌 원소의 평균값을 구함
- 행 또는 열 별 평균값을 구할 수도 있음

[형식]

```
numpy.nanmean(배열[, axis=0 또는 1 [, out=배열]])
```

- ✓ axis를 생략하면, 전체 요소의 평균값을 구함
- ✓ axis를 지정하면, 행 또는 열 별 평균값을 구함
- ✓ 0은 열 별, 1은 행 별 평균값을 구함
- ✓ out에는 결과값을 저장하기 위한 배열을 지정할 수 있음

(4) 평균값 관련 함수 | 가중 평균값 구하기

- 지정된 축을 따라 가중 평균을 구함

[형식]

```
numpy.mean(배열[, axis=0 또는 1 [, weights=배열]])
```

- ✓ axis를 생략하면, 전체 요소의 평균값을 구함
- ✓ axis를 지정하면, 행 또는 열 별 평균값을 구함
- ✓ 0은 열 별, 1은 행 별 평균값을 구함
- ✓ weights는 각 배열 요소의 가중치 배열

데이터 분석

- 주어진 배열에서 중앙값을 구함

```
import numpy as np
arr=np.array([[80,90,85],
             [75,60,80],
             [95,100,90]])
```

- 주어진 배열에서 중앙값을 구함

```
np.median(arr) → 85
```

[확인해 보기]

- 위의 배열을 1차원 배열로 만들(flatten 함수 이용)
arr.flatten() → [80, 90, 85, 75, 60, 80, 95, 100, 90]
- 1차원 배열로 변경한 배열을 오름차순으로 정렬함
np.sort(arr.flatten()) → [60, 75, 80, 80, 85, 90, 90, 95, 100]

```
np.median(arr, axis=1) → [85, 75, 95]
```

[확인해 보기]

- 주어진 배열을 정렬함

```
np.sort(arr)
```

```
array([[ 80,  85,  90],
       [ 60,  75,  80],
       [ 90,  95, 100]])
```

- 주어진 배열에서 평균값을 구함

```
np.mean(arr) 또는 np.average(arr)
```

```
83.88888888888889
```

- NaN 이 포함된 배열의 평균값 구하기

```
import numpy as np
arr=np.array([[80,np.nan,85],
             [75,60,80],
             [95,100,90]])
```

```
np.mean(arr) → nan
```

```
np.nanmean(arr) → 83.125
```

3) 분산과 표준편차 구하기

(1) 분산 구하기 | 모집단 분산

- 모집단의 분산을 구할 때 사용
- ddof 생략 또는 ddof=0 으로 설정

[형식]

```
numpy.var(배열[, axis=0 또는 1 [, ddof=0]])
```

- ✓ axis를 생략하면, 전체 요소의 분산을 구함
- ✓ axis를 지정하면, 행 또는 열 별 분산을 구함
- ✓ 0은 열 별, 1은 행 별 분산을 구함
- ✓ ddof의 기본값은 0, 1로 설정하면 표본 분산을 구함

- 표본 집단의 분산을 구할 때 사용
- ddof=1 로 설정

[형식]

```
numpy.var(배열[, axis=0 또는 1], ddof=1)
```

- ✓ axis를 생략하면, 전체 요소의 분산을 구함
- ✓ axis를 지정하면, 행 또는 열 별 분산을 구함
- ✓ 0은 열 별, 1은 행 별 분산을 구함

(2) 표준편차 구하기

- 배열의 표준편차를 구할 때 사용
- 행 또는 열 별 표준편차를 구할 수 있음

[형식]

```
numpy.std(배열[, axis=0/1])
```

- ✓ axis를 생략하면, 전체 요소의 표준편차를 구함
- ✓ axis를 지정하면, 행 또는 열 별 표준편차를 구함
- ✓ 0은 열 별, 1은 행 별 표준편차를 구함

NaN 데이터가 포함된 자료의 분산은?

→ numpy.nanvar 함수 이용

[NaN 관련 함수]

✓ NaN값 확인

`numpy.isnan(대상)`

✓ NaN을 0으로 변환

사본 생성 : `numpy.nan_to_num(대상, copy=True)`

내부 값 대체 : `numpy.nan_to_num(대상, copy=False)`

4) 상관계수와 공분산 구하기

(1) 상관계수 구하기

- 두 변수가 함께 변하는 정도를 -1~1 범위의 수로 반환
- 피어슨 상관계수(Pearson correlation coefficient)를 구함
- 상관 = 0 이면 두 변수가 독립, 즉, 한 변수의 변화로 다른 변수의 변화를 예측하지 못함
- 상관이 클수록 두 변수는 함께 많이 변화

[형식]

`numpy.corrcoef(x,y)[,]`

✓ x와y는 배열임

(2) 공분산 구하기

- 두 변수가 함께 변화하는 정도를 나타내는 지표
- 공분산이 양수이면 서로 증가 관계에 있음을 의미
- 공분산이 음수이면 서로 감소 관계에 있음을 의미
- 공분산이 0 이면 서로 독립임을 의미

[형식]

`numpy.cov(x[,y])`

✓ x와y는 배열임

2. 정렬과 집합 함수 사용하기

1) 배열 정렬 함수

(1) 배열 정렬하기

- 오름차순 정렬: `np.sort(배열)`
- 내림차순 정렬

```
np.sort(배열)[::-1]

또는
배열명[np.argsort(-배열명)]
```

- 다음 주어진 배열을 `sort` 를 이용하여 내림차순으로 정렬해 보기

```
import numpy as np
arr=np.array([4,1,5,2,6,7,8,2])
np.sort(arr)[::-1]

또는
arr[np.argsort(-arr)]
```

(2) `argsort` 함수 활용하기 | `argsort` 함수

- 배열을 정렬한 인덱스를 배열로 반환

[형식]
`numpy.argsort(배열, axis=-1)`

✓ `axis`는 정렬할 축 설정, 기본값은 마지막 축(-1), 0을 열 별 정렬, 1은 행 별 정렬

- `argsort` 함수를 이용한 정렬
 - 오름차순 정렬: `배열명[배열명.argsort()]`
 - 내림차순 정렬: `배열명[배열명.argsort()[::-1]]`
- 다음 주어진 배열을 `argsort` 를 이용하여 정렬해 보기

```
import numpy as np
arr=np.array([4,1,5,2,6,7,8,2])
arr[np.argsort(-arr)]

또는
arr[np.argsort(arr)[::-1]]

또는
arr[arr.argsort()[::-1]]
```

2) 집합 함수

(1) 집합 함수 종류

함수	설명
<code>unique(x)</code>	배열 x 내 중복된 원소를 제거한 후 정렬하여 반환
<code>intersect1d(x,y)</code>	배열 x, y의 교집합을 정렬하여 반환
<code>union1d(x,y)</code>	배열 x, y의 합집합을 정렬하여 반환
<code>in1d(x,y)</code>	배열 x가 y를 포함하는지 논리 배열로 반환
<code>setdiff1d(x,y)</code>	배열 x에서 y를 뺀 차집합을 반환
<code>setxor1d(x,y)</code>	배열 x, y의 합집합에서 교집합을 뺀 대칭차집합을 반환

3. 난수 생성하기

1) random 모듈

(1) 'random 모듈'이란?

- 특정 범위, 개수, 형태를 갖는 난수 생성
- 머신러닝을 할 때 자주 사용
- `choice`, `randint`, `rand`, `randn` 등이 있음

(2) 난수 생성하기

- 난수 발생 초기값 설정

```
numpy.random.seed
```

- 0~1 사이에서 균일 분포에서 난수 배열 생성

```
numpy.random.rand
```

- 주어진 최솟값~최댓값 사이에서 난수 배열 생성

```
numpy.random.randint 또는 numpy.random.choice
```

2) 자주 사용되는 함수

(1) choice 함수

- 기존 데이터를 sampling

[형식]

```
numpy.random.choice(x, size=None, replace=True, p=None)
```

- ✓ x는 1차원 배열 또는 정수
- ✓ X가 정수인 경우, np.arange(a)와 같은 배열 생성
- ✓ size는 추출할 데이터의 크기
- ✓ replace는 중복 허용 여부
- ✓ p는 각 데이터가 선택될 확률(1차원 배열)
- ✓ 배열 p 요소의 합은 1이며, 음수를 가질 수 없음

(2) 실습하기

- 0~10 미만인 정수 중 5개 추출(중복 허용)

```
import numpy as np
np.random.choice(10, 5, True)
```

- 0~10 미만인 정수 중 6개를 2 x 3 배열로 추출(중복 배제)

```
import numpy as np
np.random.choice(10, (2,3), False)
```

- 주어진 배열에서 6개를 2 x 3 배열로 추출

```
import numpy as np
arr=[1,2,3,4,5,6,7,8,9,10]
np.random.choice(arr, (2,3))
```

- 서로 다른 선택 확률을 가진 주어진 배열에서 4개를 추출(중복 배제)

```
import numpy as np
p=[0.2,0,0.2,0,0.05,0.1,0.2,0.05,0,0.2]
arr=[1,2,3,4,5,6,7,8,9,10]
np.random.choice(arr, 4, p=p, replace=False)
```

(3) randint 함수

- 최댓값과 최솟값 사이의 정수형 난수 생성(중복 허용)

[형식]

```
numpy.random.randint(m, n, size=정수)
```

- ✓ n는 최솟값, m은 최댓값
- ✓ size는 추출할 데이터의 크기(차원)

(4) 실습하기

- 0~10 미만인 정수 중 5개 추출

```
import numpy as np
np.random.randint(10, size=5)
```

- 5~25 미만인 정수 중 5개 추출

```
import numpy as np
np.random.randint(5, 25, size=5)
```

- 5~25 미만인 정수 중 6개를 2 x 3 배열로 추출

```
import numpy as np
np.random.randint(5, 25, size=(2,3))
```

(5) rand 함수

- 0~1 사이에서 균일 분포에서 실수형 난수 생성(중복 허용)

[형식]

```
numpy.random.rand(m, n)
```

- ✓ n, m은 추출한 데이터 수(차원)

(6) 실습하기

- 6개 추출

```
import numpy as np
np.random.rand(6)
```

- 2 x 3 배열로 추출

```
import numpy as np
np.random.rand(2,3)
```

(7) randn 함수

- 평균이 0, 표준편차가 1 인 가우시안 표준정규분포에서 실수형 난수 생성(중복 허용)

[형식]

```
numpy.random.rand(m, n)
```

✓ n, m은 추출한 데이터 수(차원)

(8) 실습하기

- 6개 추출

```
import numpy as np  
np.random.randn(6)
```

- 2 x 3 배열로 추출

```
import numpy as np  
np.random.randn(2,3)
```

8. NumPy 행렬과 벡터 활용

1. 벡터와 행렬 정의

1) 벡터와 행렬의 개념

(1) 벡터 이해하기

- 크기뿐만 아니라 방향까지 가진 수의 집합
- 벡터를 이루는 숫자 성분을 스칼라라 하고, 열 벡터와 행 벡터로 나뉨
- 선형대수에서 기본적으로 벡터를 열 벡터로 표현
- 크기가 1인 벡터를 단위벡터, 모든 원소가 0인 벡터를 영벡터라 함
- 벡터를 점과 구분하기 위해 [] (대괄호)로 둘러쌘
- array 함수를 사용하여 생성한 1차원 배열로 벡터를 나타냄

(2) 행렬 이해하기

- 크기만 가진 행과 열로 구성된 수의 집합
- array 함수를 사용하여 생성한 2차원 배열로 행렬을 나타냄
- 인덱스가 0부터 표시됨
- 행렬을 표기하기 위해서는 [] (대괄호) 안에 숫자를 나열
- $n \times n$ 행렬을 정사각 행렬,
대각선을 제외한 모든 원소가 0인 행렬을 대각 행렬,
대각선의 원소가 모두 1인 행렬을 단위 행렬 또는 항등 행렬,
주 대각선 기준으로 위 또는 아래의 모든 원소가 0인 행렬을 삼각 행렬이라 함

2) 벡터 정의하기

(1) arange 함수 이용

- 시작값과 끝값 사이에서 일정하게 떨어진 정수형 숫자들을 생성하여 배열로 반환
- 이 때, 끝값은 포함시키지 않음

[형식]

arange([start,] stop [,step])

- ✓ start와 step은 생략가능
- ✓ start의 기본값은 0, step의 기본값은 1

(2) 실습하기

- 0~10 사이의 수를 1씩 증가시켜 열 벡터를 생성하기

```
import numpy as np
cvt=np.arange(10)
cvt.shape=(len(cvt),1)
```

```
import numpy as np
cvt=np.arange(10)
cvt.shape=(len(cvt),1)
cvt
```

실행 결과

```
array([[0],
       [1],
       [2],
       [3],
       [4],
       [5],
       [6],
       [7],
       [8],
       [9]])
```

- 0~10 사이의 수를 1씩 감소시켜 행 벡터를 생성하기

```
import numpy as np
rvt=np.arange(10, 0, -1)
rvt.shape=(1, len(rvt))
```

```
import numpy as np
rvt=np.arange(10, 0, -1)
rvt.shape=(1, len(rvt))
rvt
```

결과

```
array([[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]])
```

(3) linspace 함수 이용

- 시작값과 끝값 사이에서 일정하게 떨어진 실수형 숫자들을 생성하여 배열로 반환
- 이때, 끝값을 포함하여 생성함

[형식]

`linspace(start, stop [, num] [, endpoint=True] [, restep=False])`

- ✓ start 배열의 시작값
- ✓ end 배열의 끝값
- ✓ num은 추출할 데이터 수(기본값 50)
- ✓ endpoint는 끝값 포함여부를 결정(기본값 True)
- ✓ restep은 샘플과 샘플 데이터 사이의 간격을 출력(기본값 False)

(4) 실습하기

- 1~10 사이의 수를 10개 추출하는 데 종료값을 포함시키지 않기

```
import numpy as np
np.linspace(1, 10, 10, endpoint=False)
```

```
import numpy as np
np.linspace(1, 10, 10, endpoint=False)

array([1. , 1.9, 2.8, 3.7, 4.6, 5.5, 6.4, 7.3, 8.2, 9.1])
```

- 1~10 사이의 수를 9개 추출하는 데 종료값을 포함시켜 출력하고, 샘플 사이의 간격도 출력하기

```
import numpy as np
np.linspace(1, 10, 9, restep=True)
```

```
import numpy as np
np.linspace(1, 10, 9, restep=True)

(array([ 1. , 2.125, 3.25 , 4.375, 5.5 , 6.625, 7.75 , 8.875,
        10. ]), 1.125)
```

3) 행렬 정의하기

(1) 행렬이란?

- 숫자를 직사각형 모양으로 배열해 놓은 것
- 행렬을 구성하는 숫자를 원소라 함
- 행렬의 크기: 행의 개수 x 열의 개수

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

- 하나의 행으로 구성된 행렬: 행 벡터
- 하나의 열로 구성된 행렬: 열 벡터

(2) 행렬 A

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

```
import numpy as np
A=np.array([[1,2,3], [4,5,6]])
```

(3) 행렬 B

$$B = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

```
import numpy as np
B=np.array([[1,4], [2,5], [3,6]])
```

[Tips]

(행렬 A)에서 생성한 행렬 A를 이용하여 B를 생성할 수 있음

➡ $B=A.T$ 또는 $B=A.transpose()$

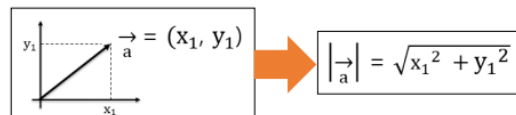
! 배열의 T 속성 또는 transpose() 메서드는 행과 열을 바꾸어 줌

2. 벡터와 행렬의 기본 연산하기

1) 벡터의 기본 연산

(1) 벡터의 크기 구하기

- 1씩 감소하는 0~10 사이의 수를 갖는 아래와 같은 행 벡터의 크기를 구하기
 - 1씩 감소하는 0~10 사이의 수를 갖는 아래와 같은 행 벡터의 크기를 구하기



```
import numpy as np
rvt=np.arange(10, 0, -1)
```

- numpy.linalg.norm 이용하기

[형식]

numpy.linalg.norm(벡터 객체명)

```
np.linalg.norm(rvt)
```

- numpy.dot 이용하기

[형식]

numpy.sqrt(numpy.dot(벡터 객체명))

```
np.sqrt(np.dot(rvt, rvt))
```

(2) 덧셈 연산하기

- + 연산자, add 함수 이용
- 다음 두 벡터의 덧셈 연산을 수행

```
import numpy as np
A=np.array([1,2])
B=np.array([5,6])
```

! + 연산자 이용: A+B

! add 함수 이용: np.add(A, B)

(3) 뺄셈 연산하기

- - 연산자, subtract 함수 이용
 - 다음 두 벡터의 뺄셈 연산을 수행
- ! - 연산자 이용: A-B
- ! subtract 함수 이용: np.subtract(A, B)

(4) 곱셈 연산하기

- * 연산자, multiply 함수 이용
 - 다음 두 벡터의 곱셈 연산을 수행
- ! * 연산자 이용: A*B
- ! multiply 함수 이용: np.multiply(A, B)

(5) 벡터의 내적 구하기

- 벡터의 내적은 dot 함수 또는 @ 연산자를 활용
 - dot 활용 방법: numpy.dot(x, y)
 - @ 연산자 활용 방법: x@y
 - 다음 두 벡터의 내적을 구하기
- ! dot 이용: np.dot(A, B)
- ! @ 이용: A@B

```
import numpy as np
A=np.array([1,2])
B=np.array([5,6])
np.dot(A, B)
```

17

(6) 벡터의 외적 구하기

- 벡터의 외적은 cross 함수를 활용
 - cross 활용 방법: `numpy.cross(x, y)`
 - 다음 두 벡터의 외적을 구하기
- ! cross 이용: `np.cross(A, B)`

```
import numpy as np
A=np.array([1,2])
B=np.array([5,6])
np.cross(A, B)

array(-4)
```

2) 행렬의 기본 연산

(1) 덧셈, 뺄셈, 곱셈 연산

- 벡터와 동일

[행렬 곱셈의 법칙]

- ✓ 교환법칙이 성립하지 않음: $A*B \neq B*A$
- ✓ 결합법칙이 성립함: $(A*B)*C = A*(B*C)$
- ✓ 배분법칙이 성립함: $(A+B)*C = A*C + B*C$
 $(A-B)*C = A*C - B*C$

(2) 행렬의 전치

- transpose 함수 또는 NumPy 배열의 T 메서드를 사용
- 원소들의 행 인덱스와 열 인덱스를 바꾼 위치로 이동시킴
- 다음 행렬의 전치행렬을 구하기

```
import numpy as np
A=np.array([[1,2,3],[4,5,6]])
```

- ! T 속성 이용: `A.T`
- ! transpose 함수 이용: `A.transpose()`

```
import numpy as np
A=np.array([[1,2,3],[4,5,6]])
A
array([[1, 2, 3],
       [4, 5, 6]])
```

(3) 항등행렬과 역행렬

- 항등행렬
 - 특정 행렬과 곱하면 특정 행렬을 그대로 반환하는 행렬
 - eye 함수를 이용
- 3x3 항등행렬을 구하기

```
import numpy as np
```

! eye 함수 이용: `I=np.eye(3)`

```
import numpy as np
I=np.eye(3)
I
```

```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

- 역행렬
 - 특정 행렬과 곱하면 항등행렬(I)이 나오는 행렬
 - $C \cdot A = I$ 또는 $A \cdot C = I$ 인 행렬 C를 행렬 A의 역행렬(A^{-1})이라 함
 - $n \times n$ 형태의 정사각행렬에서만 가능
 - 역행렬을 구할 수 없는 행렬: 특이행렬
 - `linalg.inv` 함수를 이용
- 다음 행렬 A의 역행렬을 구하기

```
import numpy as np
A=np.array([[1,2],[4,5]])
```

! inv 함수 이용: `invA=np.linalg.inv(A)`

```
import numpy as np
A=np.array([[1,2],[4,5]])
invA=np.linalg.inv(A)
invA
```

```
array([[-1.66666667,  0.66666667],
       [ 1.33333333, -0.33333333]])
```